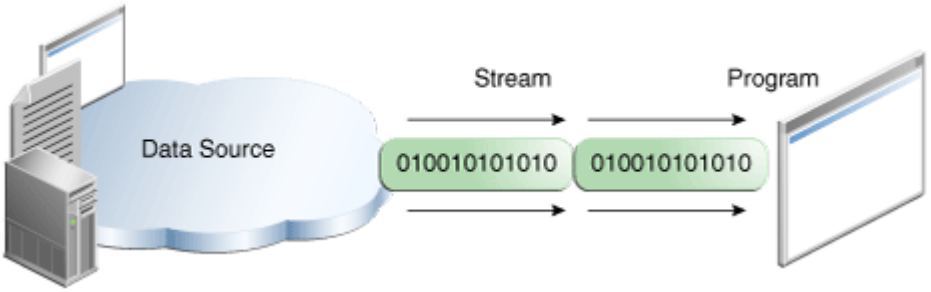
Java IO Streams

Java performs I/O through Streams. A Stream is linked to a physical layer by the Java I/O system to make input and output operations in Java. In general, a stream means a continuous flow of data. Streams are a clean way to deal with input/output without having every part of your code understand the physical.

An *I/O Stream* represents an input source or an output destination. A stream can represent many different kinds of sources and destinations, including disk files, devices, other programs, and memory arrays.

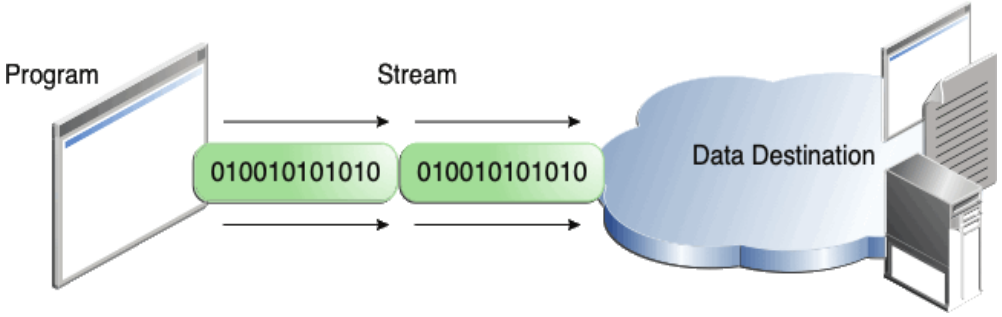
Streams support many different kinds of data, including simple bytes, primitive data types, localized characters, and objects. Some streams simply pass on data; others manipulate and transform the data in useful ways.

No matter how they work internally, all streams present the same simple model to programs that use them: A stream is a sequence of data. A program uses an *input stream* to read data from a source, one item at a time:



Reading information into a program.

A program uses an *output stream* to write data to a destination, one item at time:



Writing information from a program.

Java encapsulates Stream under the java.io package. Java defines two types of streams. They are -

Byte Stream: It provides a convenient means for handling input and output of bytes.
Character Stream: It provides a convenient means for handling input and output of characters. Character stream uses Unicode and, therefore, can be internationalized.

Java Byte Stream Classes -

Byte stream is defined by using two abstract classes at the top of the hierarchy, they are InputStream and OutputStream.

These two abstract classes have several concrete classes that handle various devices such as disk files, network connections, etc.

Some important Byte stream classes -

Stream class	Description
BufferedInputStream	Used for Buffered Input Stream.
BufferedOutputStream	Used for Buffered Output Stream.
DataInputStream	Contains a method for reading Java standard datatype
DataOutputStream	An output stream that contains a method for writing Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file.
InputStream	Abstract class that describes stream input.
OutputStream	Abstract class that describes stream output.
PrintStream	Output Stream that contain print() and println() method

These classes define several key methods. Two most important are -

read() : reads byte of data.
write() : Writes byte of data.

Java Character Stream Classes -

Character stream is also defined by using two abstract classes at the top of the hierarchy, they are Reader and Writer.

These two abstract classes have several concrete classes that handle Unicode characters.

Some important Character stream classes -

Stream class	Description
BufferedReader	Handles buffered input stream.
BufferedWriter	Handles buffered output stream.
FileReader	Input stream that reads from file.
FileWriter	Output stream that writes to file.
InputStreamReader	Input stream that translate byte to character
OutputStreamReader	Output stream that translates character to byte.
PrintWriter	Output Stream that contains print() and println() methods.
Reader	Abstract class that define character stream input
Writer	Abstract class that define character stream output

Reading Console Input -

We use the object of the BufferedReader class to take inputs from the keyboard.

Reading Characters -

read() method is used with the BufferedReader object to read characters. As this function returns integer type value, we need to use typecasting to convert it into char type.

```
int read() throws IOException
```

Below is a simple example explaining character input.

```
import java.io.*;
class CharRead
{
  public static void main( String args[]) throws IOException {
     BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
     char c = (char)br.read();   //Reading character
  }
}
```

Reading Strings in Java -

To read a string we have to use the readLine() function with the BufferedReader class's object.

```
String readLine() throws IOException
```

Program to take String input from Keyboard in Java -

```
import java.io.*;
class MyInput
{
  public static void main(String[] args) throws IOException {
```

```
        String text;
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);
        text = br.readLine();    //Reading String
        System.out.println(text);
    }
}
```

File class -

A **file class** is a class that provides methods (functions) to:

- Create a file
- Open a file
- Read data from a file
- Write data to a file
- Close a file

File myFile = new File("example.txt");

Here, `File` is a class that helps you work with files.

Program to read from a file using BufferedReader class -

```
import java.io.*;
class ReadTest
{
   public static void main(String[] args)
   {
      try
      {
         File fl = new File("d:/myfile.txt");
         BufferedReader br = new BufferedReader(new FileReader(fl)) ;
         String str;
         while ((str=br.readLine())!=null)
         {
            System.out.println(str);
         }
         br.close();
      }
      catch(IOException  e) {
         e.printStackTrace();
      }
   }
}
```

Program to write to a File using FileWriter class -

```
import java.io.*;
class WriteTest
{
  public static void main(String[] args)
  {
     try
     {
        File fl = new File("d:/myfile.txt");
        String str="Write this string to my file";
        FileWriter fw = new FileWriter(fl) ;
        fw.write(str);
        fw.close();
     }
     catch (IOException  e)
     { e.printStackTrace(); }
  }
}
```

Stream Tokenizer

StreamTokenizer is a class in Java used to **break input text into tokens**. The **Java StreamTokenizer** class takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time. The stream tokenizer can recognize identifiers, numbers, quoted strings, and various comment styles.

A *token* can be:

- Word
- Number
- Symbol (like +, -, {)
- End of line

Package:

import java.io.StreamTokenizer;

It is useful when:

- You want to **parse text files**
- You want to **read structured input**
- You are building **interpreters / compilers / parsers**

It is more flexible than Scanner in terms of customizing parsing rules.

Steps:

1. Create a Reader (FileReader / InputStreamReader)
2. Create StreamTokenizer object
3. Call nextToken() repeatedly
4. Check token type
5. Process accordingly

Field -

Following are the fields for **Java.io.StreamTokenizer** class −

ttype (token type) - Tells what type of token is read.

- **double nval** − If the current token is a number, this field contains the value of that number. Holds a numeric **value** when a token is a number.
  Eg - System.out.println(st.nval);

- **String sval** − If the current token is a word token, this field contains a string giving the characters of the word token. Holds **string value** when token is a word. Eg - System.out.println(st.sval);

- **static int TT_EOF** − A constant indicating that the end of the stream has been read.
- **static int TT_EOL** − A constant indicating that the end of the line has been read.
- **static int TT_NUMBER** − A constant indicating that a number token has been read.
- **static int TT_WORD** − A constant indicating that a word token has been read.
- **int ttype** − After a call to the nextToken method, this field contains the type of the token just read.

Example -

```
import java.io.*;

public class Main {
  public static void main(String[] args) throws Exception {
     Reader reader = new StringReader("Ekta 123 verma");
     StreamTokenizer st = new StreamTokenizer(reader);

     while (st.nextToken() != StreamTokenizer.TT_EOF) {
        if (st.ttype == StreamTokenizer.TT_WORD) {
           System.out.println("Word: " + st.sval);
        } else if (st.ttype == StreamTokenizer.TT_NUMBER) {
           System.out.println("Number: " + st.nval);
        }
```

```
    }
  }
}
```

Output:

```
Word: Ekta
Number: 123.0
Word: verma
```

Understanding nextToken() -

st.nextToken();

This method:

- Reads next token
- Updates:
  - ttype
  - sval or nval

Handling Symbols

If token is neither word nor number, ttype stores the ASCII value.

Example:

```
Reader reader = new StringReader("A + B");
StreamTokenizer st = new StreamTokenizer(reader);

while (st.nextToken() != StreamTokenizer.TT_EOF) {
  if (st.ttype == StreamTokenizer.TT_WORD) {
     System.out.println("Word: " + st.sval);
  } else {
     System.out.println("Symbol: " + (char) st.ttype);
  }
}
```

Output:

```
Word: A
Symbol: +
Word: B
```

Customizing Tokenizer Behavior

This is where StreamTokenizer becomes powerful

1. Treat characters as word

```
st.wordChars('a', 'z');
st.wordChars('A', 'Z');
```

2. Treat characters as whitespace

```
st.whitespaceChars(',', ',');
```

This tells the tokenizer: "Comma , should behave like space."

Normally, a comma is treated as a symbol.

After this rule: Ekta,123,Verma becomes like Ekta 123 Verma. So the comma gets ignored while reading tokens.

3. Enable numbers

```
st.parseNumbers();
```

This tells the tokenizer: "Recognize numbers separately." So, 123 becomes a NUMBER token instead of a word. Without this 123 may not be properly recognized as a number.

4. Make characters ordinary

```
st.ordinaryChar('/');
```

/ will be treated as symbol instead of comment

5. Handle comments

```
st.slashSlashComments(true);  // //
```

Now tokenizer ignores: // this is comment

```
st.slashStarComments(true);   /* */
```

Now tokenizer ignores: /* hello */

Example with Custom Rules

```
import java.io.*;

public class Main {
  public static void main(String[] args) throws Exception {
     Reader reader = new StringReader("Ekta,123,Verma");
     StreamTokenizer st = new StreamTokenizer(reader);

     st.whitespaceChars(',', ',');

     while (st.nextToken() != StreamTokenizer.TT_EOF) {
        if (st.ttype == StreamTokenizer.TT_WORD) {
           System.out.println("Word: " + st.sval);
```

```
        } else if (st.ttype == StreamTokenizer.TT_NUMBER) {
            System.out.println("Number: " + st.nval);
        }
    }
  }
}
```

Output:

Word: Ekta
Number: 123.0
Word: Verma

Reading from File Example -

```
import java.io.*;

public class Main {
  public static void main(String[] args) throws Exception {
    FileReader fr = new FileReader("data.txt");
    StreamTokenizer st = new StreamTokenizer(fr);

    while (st.nextToken() != StreamTokenizer.TT_EOF) {
      if (st.ttype == StreamTokenizer.TT_WORD) {
        System.out.println("Word: " + st.sval);
      } else if (st.ttype == StreamTokenizer.TT_NUMBER) {
        System.out.println("Number: " + st.nval);
      }
    }
    fr.close();
  }
}
```

Advantages -

Flexible parsing
Custom rules possible
Good for interpreters/parsers

Limitations -

Old class (legacy)
Less used today
Scanner is easier for beginners

Java Serialization and Deserialization

Serialization is a process of converting an object into a sequence of bytes which can be persisted to a disk or database or can be sent through streams. The reverse process of creating an object from a sequence of bytes is called deserialization.

A class must implement the Serializable interface present in java.io package in order to serialize its object successfully. Serializable is a marker interface that adds serializable behaviour to the class implementing it.

Java provides Serializable API encapsulated under java.io package for serializing and deserializing objects which include,

- java.io.serializable
- java.io.Externalizable
- ObjectInputStream
- ObjectOutputStream

Java Marker interface -

Marker Interface is a special interface in Java without any field and method. Marker interface is used to inform the compiler that the class implementing it has some special behaviour or meaning. Some example of Marker interface are -

- java.io.serializable
- java.lang.Cloneable
- java.rmi.Remote
- java.util.RandomAccess

All these interfaces do not have any method and field. They only add special behavior to the classes implementing them. However marker interfaces have been deprecated since Java 5, they were replaced by Annotations. Annotations are used in place of Marker Interface that play the exact same role as marker interfaces did before.

To implement serialization and deserialization, Java provides two classes ObjectOutputStream and ObjectInputStream.

ObjectOutputStream class -
It is used to write object states to the file. An object that implements java.io.Serializable interface can be written to streams. It provides various methods to perform serialization.

ObjectInputStream class -
An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

writeObject() and readObject() Methods -

The writeObject() method of the ObjectOutputStream class serializes an object and sends it to the output stream.

```
public final void writeObject(object x) throws IOException
```

The readObject() method of ObjectInputStream class references object out of stream and deserializes it.

```
public final Object readObject() throws IOException,ClassNotFoundException
```

while serializing if you do not want any field to be part of the object state then declare it either static or transient based on your need and it will not be included during the Java serialization process.

Example: Serializing an Object in Java -
In this example, we have a class that implements Serializable interface to make its object serialized.

```
import java.io.*;
class Studentinfo implements Serializable
{
  String name;
  int rid;
  static String contact;
  Studentinfo(String n, int r, String c)
  {
     this.name = n;
     this.rid = r;
     this.contact = c;
  }
}


class Demo
{
  public static void main(String[] args)
  {
     try
     {
        Studentinfo si = new Studentinfo("Abhi", 104, "110044");
        FileOutputStream fos = new FileOutputStream("student.txt");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(si);
        oos.flush();
        oos.close();
     }
     catch (Exception e)
```

```
        {
            System.out.println(e);
        }
    }
}
```

Object of Studentinfo class is serialized using writeObject() method and written to student.txt file.

Example : Deserialization of Object in Java
To deserialize the object, we are using ObjectInputStream class that will read the object from the specified file. See the below example.

```
import java.io.*;

class Studentinfo implements Serializable
{
   String name;
   int rid;
   static String contact;
   Studentinfo(String n, int r, String c)
   {
      this.name = n;
      this.rid = r;
      this.contact = c;
   }
}


class Demo
{
   public static void main(String[] args)
   {
      Studentinfo si=null ;
      try
      {
         FileInputStream fis = new FileInputStream("/filepath/student.txt");
         ObjectInputStream ois = new ObjectInputStream(fis);
         si = (Studentinfo)ois.readObject();
      }
      catch (Exception e)
      {
         e.printStackTrace(); }
      System.out.println(si.name);
      System.out. println(si.rid);
      System.out.println(si.contact);
   }
}
```

Contact field is null because,it was marked as static and as we have discussed earlier static fields does not get serialized. Static members are never serialized because they are connected to class not object of class.

NOTE -
Serialization = converting object → byte stream (for storing or sending)
Deserialization = byte stream → object
Classes used:
ObjectOutputStream
ObjectInputStream

transient Keyword

While serializing an object, if we don't want a certain data member of the object to be serialized we can mention it transient. transient keyword will prevent that data member from being serialized.

transient is a keyword used in **serialization**.

It tells Java: "Do NOT save this variable when converting object to byte stream"
If you're not using serialization → transient has no effect

Basic Syntax

transient datatype variableName;

Example -

```
class studentinfo implements Serializable
{
  String name;
  transient int rid;
  static String contact;
}
```

Making a data member transient will prevent its serialization.
In this example rid will not be serialized because it is transient, and contact will also remain unserialized because it is static.

We use transient when we don't want to store:

- Sensitive data (passwords)
- Temporary data
- Derived/calculated values

```
import java.io.*;
```

```
class User implements Serializable {
  String username;
  transient String password;

  User(String u, String p) {
    username = u;
    password = p;
  }
}


import java.io.*;

public class Main {
  public static void main(String[] args) throws Exception {
    User user = new User("Ekta", "secret123");

    // Serialization
    ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("data.txt"));
    oos.writeObject(user);
    oos.close();

    // Deserialization
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream("data.txt"));
    User u = (User) ois.readObject();
    ois.close();

    System.out.println(u.username); // Ekta
    System.out.println(u.password); // null
  }
}
```

Output Explanation

Ekta
null

Why null?

- Because password was marked as transient
- So it was NOT saved during serialization

Real-Life Use Case -

```
class BankAccount {
  String name;
  transient String pin;
}
```

When saving object:

- Name is saved
- PIN is NOT saved (security)

Practice MCQs

Q1. What is a Stream in Java?

A. A data structure
B. A sequence of elements supporting functional operations
C. A thread
D. A collection

Answer: B
Explanation: Stream is not a data structure; it processes data from collections functionally.

Q2. Which package contains Stream API?

A. java.util
B. java.io
C. java.util.stream
D. java.lang

Answer: C
Explanation: All Stream classes and interfaces are in `java.util.stream`.

Q3. Streams are:

A. Mutable
B. Reusable
C. Immutable and single-use
D. Stored in memory

Answer: C
Explanation: Streams cannot be reused once consumed.